A Constructive Realisation of Graphic Equalisation

Deterministic Proposal Closure, Boundary Interaction, and Reference Execution Semantics

Matthew Parslow
Independent Researcher

Draft — draft-2026-09-06.1
6 September 2026 • document revision 8

DRAFT — draft-2026-09-06.1r8 (6 September 2026). This document is under active revision and is circulated for comment, not as a finished result. Definitions, numbering and claims may change between drafts; cite the version string above if referring to it. The finite-profile proof and the conformance conditions of Section 20 are the intended points of attack.

Abstract

The companion theory of Graphic Equalisation is stated extensionally: a realised graph induces admissible continuations, interaction constrains those continuations, and collapse produces realised history. The theory does not require a particular executor.

This paper gives one constructive reference realisation. It specifies synchronous rule closure over an immutable antecedent base, provisional identity handling, typed well-formedness predicates, deterministic compatible merge, local and world-directed target decomposition, optional represented valuation, boundary exposure, provenance-preserving residual construction, and conditions for order-independent least-fixed-point closure. The construction is both executable semantics and experimental apparatus: implementation forced ambiguous prose to resolve into explicit causal choices.

No claim is made that nature literally executes these algorithms. The claim is that, under the stated conditions, the finite core satisfies the stated determinism constraint; the optional interfaces require additional declared policies.

1 Separation from the theory

Reading for the claim. What this construction buys is one executable semantics that forced the theory's ambiguities to resolve into explicit causal choices. As with the companion theory, the components used here are individually established and cited as such; the claim is their arrangement, which is not reachable by evaluating those components separately, since they are stipulated to be known. The construction inherits the extensional theory's composition challenge. Its own conformance claim fails if a finite-profile execution depends on rule enumeration, reads a same-round addition, or commits a boundary family rejected by its stated well-formedness predicate.

The theory defines

$$G_t \to \mathcal{P}(G_t) \xrightarrow{\mathcal{C}_t} G_{t+1}.$$

Section 5 constructs a finite executable core. The optional decision, publication and exposure interfaces below are extension contracts requiring supplied policies, not further closed algorithms. The core constructs one finite executable representation of $\mathcal{P}(G_t)$ and one deterministic graph/world execution semantics. Proposal records, synchronous rounds, provisional identifiers,

explicit merge functions, FAIL values, causal collection cuts, and executor tie handling are implementation-specific.

The distinction follows a standard separation in semantics between an extensional relation and one operational procedure that computes or realises it. Fixed-point semantics for recursive rule systems provide a particularly close computational precedent (Tarski, 1955; Cousot and Cousot, 1977; Ceri et al., 1989).

2 Graph deltas and rewrite semantics

Let

$$G = (g, r, \Gamma)$$

and let a realised graph delta be

$$\Delta G = (\Delta G^+, \Delta G^-).$$

Define

$$G \oplus \Delta G = (G \setminus \Delta G^-) \cup \Delta G^+$$

when $\mathrm{WF}(G, \Delta G)$ holds.

A software implementation may expose

$$\mathrm{AtomicApply}(G, \Delta G) = \begin{cases} G \oplus \Delta G, & \mathrm{WF}(G, \Delta G), \\ \mathrm{FAIL}, & \text{otherwise}, \end{cases}$$

but FAIL is an implementation result, not a theory-level state.

This extensional add/remove representation sits within the broad tradition of algebraic graph transformation, where graph rewriting and its applicability conditions are given explicit mathematical semantics (Corradini et al., 1997; Ehrig et al., 2006).

3 ADD/REMOVE completeness

Typed mutations reduce extensionally to

$$\Pi_G = \{\mathrm{ADD}, \mathrm{REMOVE}\}.$$

Remark 3.1 (Finite add/remove representation). For every pair of finite graph states G and H, a finite sequence of ADD and REMOVE operations transforms G into H.

Proof. Remove every represented fact in $G \setminus H$ and add every represented fact in $H \setminus G$. □

The set identity is intentionally weak: it establishes representational completeness for finite extensional graph state, not efficiency, confluence, or semantic preservation.

4 Guarded proposals

A reference proposal is

$$\pi = \nu \mathbf{a}.(P_G, P_{\mathcal{B}}, \mathcal{D}),$$

where P_G is a source-local antecedent condition, $P_{\mathcal{B}}$ an optional boundary-language precondition, $\mathbf{a}$ a finite set of provisional identities, and $\mathcal{D}$ a finite transformation term.

Proposal objects are implementation artefacts representing possible deformations. They are not substrate primitives.

5 A finite executable core

Fix a finite constant set A, a finite typed relational vocabulary, and a finite set U of possible ground proposal records. A graph is represented by node facts $\mathrm{Node}(a)$ and edge facts $\mathrm{Edge}_t(a, b)$, plus facts encoding the fixed rules r and grammar Γ for this invocation. Every edge endpoint must have a node fact. Frames are selected edge types and boundary targets, not an additional primitive (theory Definitions 3.1, 6.2).

Each rule is a finite positive Horn clause. Its body reads base facts or positive proposal facts; its head emits one keyed record $u = (k, \tau, P^+, P^-)$ with finite addition and removal sets. The key is (rule identifier, ground binding), so repeating the same derivation emits the same record. In this profile there is no unbounded name invention: any provisional identities are preallocated members of A. Negation reads only the immutable base, if used. Proposed removal does not delete a premise during closure. These restrictions make ground rule instances finite.

Put $L = 2^U$, ordered by inclusion, with bottom $\varnothing$ and total join $\sqcup_D = \cup$. For $D \in L$,

$$\mathrm{Frontier}^+(D) = \bigcup_{u \in D} P_u^+, \qquad \widetilde{g}(D) = g^{\mathrm{base}} \cup \mathrm{Frontier}^+(D).$$

This union is a symbolic matching view, never a realised commit. Define

$$F(D) = D \cup \{u \in U : \text{a ground rule body is true in } \widetilde{g}(D) \text{ and emits } u\}.$$

A round evaluates all enabled instances on this fixed view. Incompatibility is retained as proposal data; it cannot make the lattice join partial.

For a boundary target τ, collect D_τ and let $P_\tau^\pm = \bigcup_{u \in D_\tau} P_u^\pm$. Set $\mathrm{WF}(g, P_\tau)$ true exactly when additions and removals are disjoint, every fact has the declared type and arity, every removal exists in the base, and the candidate graph $(g \setminus P_\tau^-) \cup P_\tau^+$ has all edge endpoints and satisfies the explicitly supplied finite grammar predicates. Realisation returns

$$\Delta_\tau^R = \begin{cases} (P_\tau^+, P_\tau^-), & \mathrm{WF}(g, P_\tau), \\ (\varnothing, \varnothing), & \text{otherwise.} \end{cases}$$

Thus the policy rejects the whole conflicting boundary family, independent of enumeration order. Its transient witness records every contributing key, the predicate failures and acceptance status. Applying Δ_τ^R constructs the collapse step; comparing it with each proposal constructs the residual (theory Definitions 3.17, 4.1, 8.1). Rejection is this executor's policy, not the only admissible grounding.

The typed predicates below specialise as follows in this core: $\mathrm{WF}_{\mathrm{sym}}$ checks fact types and disjoint effects before substituting preallocated names; $\mathrm{WF}_{\mathrm{app}}$ is realised WF after substitution; $\mathrm{WF}_{\mathrm{rel}}$ checks a supplied finite boundary translation; $\mathrm{WF}_{\mathrm{seq}}$ checks dependency ordering; $\mathrm{WF}_{\mathrm{sym}}^W$ repeats the symbolic check on a world base; and $\mathrm{WF}_{\mathrm{decision}}$ combines these checks for a selected family. A translation is a declared finite table; dependency ordering is acyclicity of the declared dependency graph. Valuation and prospect functions are optional supplied functions, absent from this minimal profile.

For example, from $g = \{\mathrm{Node}(a), \mathrm{Node}(b)\}$, a rule emits u_1 adding $\mathrm{Edge}(a, b)$; a second rule matches that positive proposal fact and emits u_2 adding $\mathrm{Edge}(b, a)$. The successive proposal sets are $\varnothing$, $\{u_1\}$, $\{u_1, u_2\}$ and then stable. Both edges are committed together at their common target if its grammar permits them. A proposal also removing $\mathrm{Edge}(a, b)$ conflicts and causes this boundary family to be rejected, while the proposal lattice remains intact.

6 Proposal construction function

The formal implementation replaces the code-oriented name `run_rules` with

$$\boxed{\mathrm{ProposalClosure}(g^{\mathrm{base}}, r, \Gamma) = D^*.}$$

A concrete codebase may retain any convenient function name.

For one invocation,

$$g^{\text{base}} = g_t^I$$

is immutable. Let

$$D^{(0)} = \varnothing$$

and define

$$\text{Frontier}^+(D) = \{f : f \text{ is a completed positive proposal fact in } D\}.$$

Here $\varnothing$ is an ordinary value of the instantiated set-valued closure dimension: it means "the current proposal family contains no members." It is not the theory's NULL/non-existence state.

The symbolic matching view is

$$\widetilde{g}^{(k)} = g^{\text{base}} \boxplus \text{Frontier}^+(D^{(k)}), \qquad g \boxplus F \neq g \oplus F.$$

7 Synchronous rounds and antecedence

Every rule in round k reads the same $\widetilde{g}^{(k)}$ and writes to a fresh $E^{(k)}$. Same-round writes are not visible. After the round,

$$D^{(k+1)} = D^{(k)} \sqcup_D E^{(k)},$$

with total set union during closure; incompatibility is recorded and handled only by the realisation policy of Section 5.

This is one constructive realisation of the theory-level antecedent-state principle. Comparable staged semantics appear in rule evaluation and deductive databases, where immediate-consequence operators repeatedly extend a fact set until closure (Ceri et al., 1989).

8 Least-fixed-point closure

Let

$$F_{g^{\text{base}},r,\Gamma}$$

be the synchronous proposal operator.

Theorem 8.1 (Finite order-independent proposal closure)**.** *For the finite positive profile of Section 5, synchronous iteration $D^{k+1} = F(D^k)$ from $D^0 = \varnothing$ reaches $\text{lfp}(F)$ after at most $|U|$ strict growth rounds. The result is independent of the enumeration of ground rule instances within each round.*

Proof. Positive bodies remain true as D grows, so F is monotone and inflationary on the finite complete lattice 2^U. Each nonstationary round adds a member of U, hence there are at most $|U|$ strict rounds. At stationarity the set is a fixed point. For any fixed point X, induction from $D^0 \subseteq X$ and monotonicity gives $D^k \subseteq X$; the stationary set is therefore least. A round's union depends only on its enabled set, not its enumeration. Realisation conflicts are handled after closure and cannot invalidate this lattice argument. □

The existence of least fixed points for monotone operators on complete lattices is classical (Tarski, 1955); fixed-point semantics are foundational in denotational semantics and abstract interpretation (Cousot and Cousot, 1977), and least-fixed-point evaluation is standard for positive recursive rule systems (Ceri et al., 1989).

The proof applies to the finite profile just defined. Extensions with recursive fresh-name invention need a new termination argument; a monotone operator alone does not imply finite termination.

9 Removal and freshness stratification

Proposed removals remain effects during monotone positive closure and do not retract premises that enabled their own derivation in the same unstratified closure.

A fresh-name minting site may not become recursively self-enabling solely through identities minted from that site.

These restrictions preserve the monotone conditions under which the least-fixed-point construction in Theorem 8.1 applies. Non-monotone rule languages require additional semantics rather than being silently treated as positive closure.

10 Typed well-formedness

The executor distinguishes:

$\mathrm{WF}(g, \Delta)$
: realised delta validity.

$\mathrm{WF}_{\mathrm{sym}}(g, S)$
: source-symbolic coherence before identity resolution.

$\mathrm{WF}_{\mathrm{app}}(g, S)$
: existence of a valid realised local image.

$\mathrm{WF}_{\mathrm{rel}}(S)$
: coherent translation to boundary vocabulary.

$\mathrm{WF}_{\mathrm{seq}}(S^L, S^A)$
: coherent local/outward sequencing.

$\mathrm{WF}^W_{\mathrm{sym}}(W, S)$
: world-side symbolic coherence.

$\mathrm{WF}_{\mathrm{decision}}(g, S)$
: joint source-decision validity.

These predicates validate different object types and cannot be substituted merely because they share a name. Typed application conditions and gluing constraints are likewise central in algebraic graph transformation (Corradini et al., 1997; Ehrig et al., 2006).

11 Targets and atomicity

Each effect has one target:

$$\tau : V \to \{\mathrm{local}\} \cup \{\mathrm{world}_k : W_k \in \mathcal{W}(G)\}.$$

Mixed-target families are decomposed into same-target candidates connected by dependency edges.

Proposition 11.1 (Boundary-scoped atomicity)**.** *In this reference semantics, one primitive atomic realised commit is scoped to one realised boundary. Effects that must succeed or fail together must therefore either be represented at a common realised boundary or be implemented by an additional coordination protocol.*

This is not a claim that distributed atomicity is impossible in computer systems. Transaction-processing systems explicitly construct distributed atomicity by adding commit protocols, logging, recovery, and coordination machinery (Gray and Reuter, 1993). The proposition states that such machinery is *additional represented structure*; distributed atomicity is not free at the primitive boundary layer.

12 Boundary identity publication

For source-private identity x needed at boundary $\mathcal{B}_k$,

$$\eta_{G\to\mathcal{B}_k} : \mathrm{Id}_G \rightharpoonup \mathrm{Handle}_{\mathcal{B}_k}.$$

Then

$$\text{realise } x \prec \text{publish } h_x \prec \text{expose dependent deformation.}$$

An identity may be public in one boundary vocabulary and private in another.

The ordering is causal rather than merely chronological. Lamport's "happened-before" relation and event-structure semantics are standard precedents for representing causal partial order without requiring a single global time sequence (Lamport, 1978; Winskel, 1987).

13 Optional source decision

The theory does not require valuation. For graphs that represent alternatives, define readiness at causal collection cut $\chi_{G,t}$:

$$\mathrm{ready}_{G,t}(\pi) \iff \forall d \in \mathrm{Prereq}(\pi), \mathrm{resolved}(d) \preceq \chi_{G,t}.$$

A feasible family

$$\mathfrak{F}^{\mathrm{decision}}_{G,t}$$

is formed from represented admissibility and typed well-formedness.

Where valuation exists,

$$s^{\mathrm{decision}}_{G,t}(S) = \mathrm{Value}\left(S^L, \mathrm{Prospect}_{G,t}(S^A)\right),$$

and

$$\mathcal{M}^{\mathrm{decision}}_{G,t} = \arg\max_{S\in\mathfrak{F}^{\mathrm{decision}}_{G,t}} s^{\mathrm{decision}}_{G,t}(S).$$

The deterministic reference executor does not break exact ties by hidden randomness or enumeration order.

14 Relative weighting

Where the optional decision layer uses weights, those weights must follow the theory's relative-weighting discipline. No operation possesses an absolute substrate weight.

For candidate deformations δ_i, δ_j in the same instantiated comparison context, define

$$\omega_{\mathcal{F},t}(\delta_i, \delta_j) = w_{\mathcal{F},t}(\delta_i) - w_{\mathcal{F},t}(\delta_j).$$

A scalar representation w is only a gauge representative:

$$w_{\mathcal{F},t}(\delta) \sim w_{\mathcal{F},t}(\delta) + c$$

for any common offset c applied to the active comparison family.

An inadmissible or absent candidate is not assigned 0, $-\infty$, or any other extreme weight; its weight dimension is not instantiated in that comparison.

15 Component-scoped ties

A represented coupling graph records dependency, incompatibility, non-factorisable admissibility, prospect interaction, or valuation interaction.

Independent components resolve separately only when feasibility, prospect composition, and valuation genuinely factorise. Hence

$$\boxed{\text{tie scope} = \text{represented decision-coupling scope.}}$$

This is a deterministic decomposition rule. Re-evaluation without changed represented state does not break an exact tie.

16 World-directed deformation

A selected outward candidate is translated into boundary language by exposure:

$$\bar{\pi} = \mathrm{Expose}_G(\pi).$$

Exposure is event-local. The reference executor creates no substrate-level persistent request object with intrinsic pending, retry, cancellation, acknowledgement, or expiry.

Persistence, where required, is explicit graph/world state. Message queues, leases, retries, acknowledgements, and transaction protocols may all be added as represented structures; they are not implicit in exposure itself.

17 World interaction and determinism

At world interaction event e,

$$B_{W,e} = B^{\mathrm{self}}_{W,e} \cup \bigcup_j B^{\mathrm{exposed}}_{G_j \to W,e} \cup B^{\mathrm{external}}_{W,e}.$$

A graph-world recursively uses deterministic resolution. More generally, the reference implementation assumes that a causally complete antecedent configuration determines the realised successor, as a design assumption under the Graph Law schema and the theory's separate determinism commitment.

Remark 17.1 (The antecedent base is present structure). "Antecedent" here names the immutable base a round reads, not a retained temporal history. Consistently with how the Graph Law treats history, prior rounds influence a closure only where their consequences survive in the base being read; the executor holds no stack of earlier graph states and requires none. Reproducibility is therefore stated over causally sufficient *present* structure — identical bases yield identical realised successors — and not over an agreement of histories.

An implementation may nevertheless attach a stochastic *observation model* to an incompletely represented external frame. Such a model expresses uncertainty over unresolved determining dimensions:

$$\boxed{\text{stochastic observation model} \neq \text{stochastic substrate transition.}}$$

No hidden random source may be inserted into an executor that is claimed to implement the deterministic Graph Law. If a deliberately stochastic simulator is used for convenience, that randomness is an explicit extra modelling assumption rather than part of the reference semantics.

18 Provenance and residual

Boundary merge returns realised delta plus a transient provenance witness.

For proposed and realised deltas

$$\Delta^P = (P^+, P^-), \qquad \Delta^R = (R^+, R^-),$$

define

$$\boxed{\Delta^R \ominus \Delta^P = \left(R^+ \setminus P^+, R^- \setminus P^-, P^+ \setminus R^+, P^- \setminus R^-\right).}$$

If

$$\Delta^R = (\varnothing, \varnothing),$$

the residual contains the entire unrealised proposal image. Here the empty delta is again a legitimate value of an instantiated structure rather than NULL (Section 6).

Residual is mismatch, not diagnosis. Provenance merely permits the implementation to attribute that mismatch to contributing proposal sources.

19 Input integration

Reference input is

$$I_t = (d_t, \epsilon_t, K_t),$$

separating direct data, residual, and represented precision/freshness metadata.

Integrate_G produces the post-input antecedent base for the next closure. World-caused integration is not checked against self-admissibility as though the graph could veto the world.

The distinction between externally caused input and internally selected output is part of the theory; the tuple above is one implementation encoding of it.

20 Conformance tests

A reference implementation should test:

1. enumeration-order invariance under the assumptions of Theorem 8.1;
2. same-round non-readability;
3. monotone frontier growth where monotonicity is claimed;
4. removal and freshness stratification;
5. deterministic compatible merge;
6. correct boundary identity publication;
7. absence of accidental cross-boundary atomicity;
8. stable exact-tie behaviour;
9. gauge invariance of relative weighting under common offsets;
10. absence of a numeric weight for candidates outside the comparison domain;
11. provenance-preserving residual;
12. full-proposal residual under complete non-realisation;

13. frame-local termination;
14. absence of hidden executor stochasticity;
15. reproducibility of realised successors for identical causally complete antecedent configurations.

21 What implementation taught the theory

The following design-history distinctions guide this executor and proposed replacements; they are not theorems about every possible executor:

1. antecedent state is distinct from consequences of the current transition;
2. self-admissibility cannot function as armour against world-caused change;
3. outward action and inbound observation are different causal legs;
4. identity must be frame-typed;
5. failed interaction requires no persistent attempt object;
6. residual is mismatch and need not contain explanation;
7. previous failure changes later dynamics only if consequences survive in state;
8. distinct boundaries cannot be silently treated as one primitive atomic transaction;
9. higher-frame quantities may be first-class without being primitives of the lower graph;
10. absent dimensions must not be represented by zero or an extreme numeric sentinel;
11. apparent stochasticity belongs to incomplete observation, not to the deterministic Graph Law.

22 Positioning and prior art

Graph rewriting. Algebraic graph transformation supplies mature semantics for graph rewrite rules, applicability, conflicts, and compositionality (Corradini et al., 1997; Ehrig et al., 2006). The present executor uses a simpler extensional ADD/REMOVE representation because its purpose is to realise the companion theory, not to replace the graph-transformation literature.

Fixed-point semantics. Knaster–Tarski supplies least and greatest fixed points for monotone operators on complete lattices (Tarski, 1955). Abstract interpretation and deductive-database semantics make fixed-point construction operational in program analysis and recursive rule evaluation (Cousot and Cousot, 1977; Ceri et al., 1989). The present contribution is the particular decomposition into immutable antecedent base, positive symbolic frontier, typed proposal effects, deterministic merge, and boundary execution required by Graphic Equalisation.

Causality and concurrency. Lamport's event ordering and Winskel's event structures provide standard models in which causal precedence is a partial order rather than a global clock (Lamport, 1978; Winskel, 1987). The reference executor's causal cuts and publication-before-exposure ordering belong to that tradition.

Atomicity and distributed transactions. Transaction processing demonstrates that atomicity across distributed resources can be constructed, but only by adding protocols and recovery structure (Gray and Reuter, 1993). This is consistent with Proposition 11.1: cross-boundary atomicity is representable, but it is not primitive.

Novelty. The individual implementation techniques are therefore largely established. The contribution of this paper is their disciplined composition as one reference realisation of the Graphic Equalisation substrate, plus the record of which theoretical distinctions became necessary when the implementation was forced to close.

23 Conclusion

This construction is one way to compute and realise Graphic Equalisation. It is intentionally more specific than the theory:

$$\boxed{\text{theory} \rightarrow \text{constructive implementation} \rightarrow \text{edge cases} \rightarrow \text{stronger theory.}}$$

The implementation is both an executable reference and an experimental apparatus for formal clarification, without being mistaken for the ontology it helped reveal.

References

Stefano Ceri, Georg Gottlob, and Letizia Tanca. *What You Always Wanted to Know About Datalog (And Never Dared to Ask).* IEEE Transactions on Knowledge and Data Engineering, 1(1):146–166, 1989.

Andrea Corradini, Ugo Montanari, Francesca Rossi, Hartmut Ehrig, Reiko Heckel, and Michael Löwe. Algebraic approaches to graph transformation, Part I: Basic concepts and double pushout approach. In Grzegorz Rozenberg, editor, *Handbook of Graph Grammars and Computing by Graph Transformation, Volume 1: Foundations*, pages 163–245. World Scientific, 1997.

Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*, pages 238–252, 1977.

Hartmut Ehrig, Karsten Ehrig, Ulrike Prange, and Gabriele Taentzer. *Fundamentals of Algebraic Graph Transformation.* Springer, Berlin, 2006.

Jim Gray and Andreas Reuter. *Transaction Processing: Concepts and Techniques.* Morgan Kaufmann, San Mateo, 1993.

Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, 1978.

Alfred Tarski. A lattice-theoretical fixpoint theorem and its applications. *Pacific Journal of Mathematics*, 5(2):285–309, 1955.

Glynn Winskel. Event structures. In Wilfried Brauer, Wolfgang Reisig, and Grzegorz Rozenberg, editors, *Petri Nets: Applications and Relationships to Other Models of Concurrency*, Lecture Notes in Computer Science 255, pages 325–392. Springer, 1987.